Git Working On Commands

09 August 2026 13:40

Git Complete Practical Notes — Commands + Usage + Output

1. CONFIGURE GIT

$ git config --global user.name deepak90221
OUTPUT:
(no output)
Meaning:
Sets your Git username.
$ git config --global user.email 116907628+deepak90221@users.noreply.github.com
OUTPUT:
(no output)
Meaning:
Sets your Git email.
$ git config --list
OUTPUT:
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=schannel
core.autocrlf=true
credential.helper=manager
user.name=deepak90221
user.email=116907628+deepak90221@users.noreply.github.com
...
Meaning:
Displays your Git configuration.
2. CREATE PROJECT FOLDER

$ cd ~/Documents
OUTPUT:
(no output)
Meaning:
Moves into the Documents folder.
$ mkdir git-begin
OUTPUT:
(no output)
Meaning:
Creates a folder named git-begin.
$ cd git-begin
OUTPUT:
(no output)
Meaning:
Enters the git-begin folder.
3. INITIALIZE GIT

$ git init
OUTPUT:
Initialized empty Git repository in
C:/Users/deepa/Documents/git-begin/.git/
Meaning:
Creates the hidden .git folder and turns the project into a Git repository.
$ ls -a
OUTPUT:
./
../
.git/

Meaning:
Shows normal and hidden files. .git is Git's repository folder.
4. CREATE PROJECT FILES

$ touch index.html style.css
OUTPUT:
(no output)
Meaning:
Creates index.html and style.css.
$ ls
OUTPUT:
index.html
style.css
Meaning:
Shows the files inside the project.
5. CHECK STATUS

$ git status
OUTPUT:
On branch master
No commits yet
Untracked files:
  index.html
  style.css
Meaning:
Git sees the files but is not tracking them yet.
6. ADD FILES TO STAGING

$ git add index.html style.css
OUTPUT:
(no output)
Meaning:
Moves both files into the staging area.
$ git status
OUTPUT:
On branch master
No commits yet
Changes to be committed:
  new file: index.html
  new file: style.css
Meaning:
The files are staged and ready to commit.
7. FIRST COMMIT

$ git commit -m "add index.html and style.css"
OUTPUT:
[master (root-commit) 628fead] add index.html and style.css
 2 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 index.html
 create mode 100644 style.css
Meaning:
Creates the first saved snapshot of the project.
8. OPEN PROJECT IN VS CODE

$ code .
OUTPUT:
(no terminal output)
Meaning:
Opens the current project folder in Visual Studio Code.
9. MODIFY A FILE

After editing index.html:
$ git status
OUTPUT:
On branch master
Changes not staged for commit:
  modified: index.html
Meaning:
Git detected that index.html changed after the previous commit.
10. VIEW UNSTAGED CHANGES

$ git diff
OUTPUT:
diff --git a/index.html b/index.html
index e69de29..efc3d43 100644
--- a/index.html
+++ b/index.html
@@ -0,0 +1,5 @@
+<html>
+   <body>
+   <h1>This is git tutorial</h1>
+   </body>
+</html>
Meaning:
Shows exactly what changed in the working file.
11. STAGE THE CHANGE

$ git add .
OUTPUT:
(no output)
Meaning:
Stages all changed/untracked files.
$ git status
OUTPUT:
On branch master
Changes to be committed:
  modified: index.html
Meaning:
index.html is now staged.
12. VIEW STAGED CHANGES

$ git diff --staged
OUTPUT:
diff --git a/index.html b/index.html
...
+<html>
+   <body>
+   <h1>This is git tutorial</h1>
+
+   <h2>Learn asap !</h2>
+
+   </body>
+</html>
Meaning:
Shows changes that are already staged for the next commit.
13. COMMIT THE CHANGE

$ git commit -m "add html content"
OUTPUT:
[master 85fc633] add html content
 1 file changed, 8 insertions(+)
Meaning:
Saves the staged HTML changes as a new commit.
14. CHECK CLEAN STATUS

$ git status
OUTPUT:
On branch master
nothing to commit, working tree clean
Meaning:
There are no pending changes.
15. MODIFY STYLE.CSS

After modifying style.css:
$ git status
OUTPUT:
On branch master
Changes not staged for commit:
  modified: style.css
Meaning:
style.css was changed but not staged.
16. COMMIT TRACKED FILE DIRECTLY

$ git commit -a -m "add styles"
OUTPUT:
[master 07ff967] add styles
 1 file changed, 3 insertions(+)
Meaning:
Stages and commits modified/deleted tracked files in one command.
IMPORTANT:
-a does NOT include brand-new untracked files.
17. DELETE A FILE

After deleting style.css:
$ git status
OUTPUT:
On branch master
Changes not staged for commit:
  deleted: style.css
Meaning:
Git detected that style.css was deleted.
$ git add .
OUTPUT:
(no output)
$ git status
OUTPUT:
On branch master
Changes to be committed:
  deleted: style.css
Meaning:
The deletion is now staged.
18. MOVE/RENAME FILE

After moving style.css into css/:
$ git add .
OUTPUT:
(no output)
$ git status
OUTPUT:
Changes to be committed:
  renamed: style.css -> css/style.css
Meaning:
Git recognized the file movement as a rename.
19. ADD .gitignore

After creating css/.gitignore:
$ git add .
OUTPUT:
(no output)
$ git status
OUTPUT:
Changes to be committed:
  new file: css/.gitignore
  renamed: style.css -> css/style.css
Meaning:
Both the rename and new .gitignore are staged.
20. COMMIT

$ git commit -m "add gitigonore"
OUTPUT:
[master ca514f0] add gitigonore
 2 files changed, 2 insertions(+)
 create mode 100644 css/.gitignore
 rename style.css => css/style.css (100%)
Meaning:
Saves the rename and .gitignore changes.
21. RESTORE FILE

$ git restore css/style.css
OUTPUT:
(no output)
Meaning:
Restores the file to its committed version.
If the file was already identical to the committed version,
nothing visibly changes.
22. VIEW COMMIT HISTORY

$ git log --oneline
OUTPUT:
ca514f0 (HEAD -> master) add gitigonore
07ff967 add styles
85fc633 add html content
628fead add index.html and style.css
Meaning:
Shows commit IDs and commit messages in a short format.
23. CREATE BRANCH

$ git branch welcome
OUTPUT:
(no output)
Meaning:
Creates a branch called welcome.
$ git branch
OUTPUT:
* master
  welcome
Meaning:
Shows all branches.
* indicates the current branch.
24. SWITCH BRANCH

$ git switch welcome
OUTPUT:
Switched to branch 'welcome'
Meaning:
Moves from master to welcome.
25. CREATE + SWITCH TO NEW BRANCH

$ git switch -c temp
OUTPUT:
Switched to a new branch 'temp'
Meaning:
Creates temp and immediately switches to it.
26. CHECK BRANCHES

$ git branch
OUTPUT:
  master
  welcome
* temp
Meaning:
Shows all branches and the current branch.
27. SWITCH BACK

$ git switch welcome
OUTPUT:
Switched to branch 'welcome'
Meaning:
Moves to the welcome branch.
28. SWITCH TO MASTER

$ git switch master
OUTPUT:
Switched to branch 'master'
Meaning:
Moves back to master.
29. MERGE BRANCH

$ git merge welcome
OUTPUT:
Updating ca514f0..6e6c7e6
Fast-forward
 css/.gitignore | ...
 css/style.css => style.css | ...
 ...
Meaning:
Combines welcome branch changes into master.
Fast-forward means master simply moves forward to the newer commit
because there were no conflicting/diverging master changes.
30. VIEW MERGED HISTORY

$ git log --oneline
OUTPUT:
6e6c7e6 (HEAD -> master, welcome) add welcome mssge
ca514f0 (temp) add gitigonore
07ff967 add styles
85fc633 add html content
628fead add index.html and style.css
Meaning:
Shows that master now contains the welcome commit.
31. DELETE BRANCHES

$ git branch -d welcome
$ git branch -d temp
OUTPUT:
Deleted branch welcome (was 6e6c7e6).

Deleted branch temp (was ca514f0).
Meaning:
Deletes branches that are no longer needed.
32. CREATE GITHUB REPOSITORY

On GitHub:
Create repository
↓
Choose repository name
↓
Make it Public
↓
Create repository
↓
Copy repository URL

Example:
<repo_link>
33. CONNECT LOCAL REPOSITORY TO GITHUB

$ git remote add origin <repo_link>
OUTPUT:
(no output)
Meaning:
Connects your local Git repository to the GitHub repository.
origin = name given to the remote repository.
34. CHECK REMOTE

$ git remote -v
OUTPUT:
origin <repo_link> (fetch)
origin <repo_link> (push)
Meaning:
Shows the remote repository connected to your project.
35. INITIALIZE IF REQUIRED

$ git init
OUTPUT:
Reinitialized existing Git repository in
C:/Users/deepa/Documents/git-begin/.git/
Meaning:
Initializes Git.
IMPORTANT:
If you already used git init, you normally DON'T need to run it again.
36. ADD FILES

$ git add .
OUTPUT:
(no output)
Meaning:
Stages all changes.
37. COMMIT

$ git commit -m "first commit"
OUTPUT:
[master XXXXXXX] first commit
 ...
Meaning:
Creates a commit containing the staged changes.
38. RENAME MASTER TO MAIN

$ git branch -m main
OUTPUT:
(no output)
Meaning:
Renames the current branch from master to main.
39. RENAME CURRENT BRANCH TO MASTER

$ git branch -m master
OUTPUT:
(no output)
Meaning:
Renames the current branch to master.
40. PUSH MAIN TO GITHUB

$ git push -u origin main
OUTPUT:
Enumerating objects: ...
Counting objects: ...
Writing objects: ...
To <repo_link>
 * [new branch]   main -> main
branch 'main' set up to track 'origin/main'
Meaning:
Uploads your local main branch to GitHub.
-u:
Sets origin/main as the upstream branch.
41. PUSH MASTER TO GITHUB

$ git push -u origin master
OUTPUT:
Enumerating objects: ...
Counting objects: ...
Writing objects: ...
To <repo_link>
 * [new branch]   master -> master
branch 'master' set up to track 'origin/master'
Meaning:
Uploads your local master branch to GitHub.
42. FORCE PUSH

$ git push -u -f origin main
OUTPUT:
...
 + oldcommit...newcommit main -> main (forced update)
Meaning:
Forcefully replaces remote branch history with your local history.
WARNING:
Do NOT use -f unless you intentionally want to overwrite remote history.
43. PULL

$ git pull
OUTPUT:
Already up to date.
OR:
Updating XXXXXXX..YYYYYYY
Fast-forward
 ...
Meaning:
Downloads changes from the remote repository and merges them into your
current local branch.

44. FETCH

$ git fetch
OUTPUT:
(no output)
OR:
From <repo_link>
  XXXXXXX..YYYYYYY main -> origin/main
Meaning:
Downloads remote changes but does NOT merge them into your current branch.
45. CLONE

$ git clone <repo_link>
OUTPUT:
Cloning into 'repository-name'...
remote: Enumerating objects: ...
Receiving objects: 100% ...
Resolving deltas: 100% ...
Meaning:
Downloads an entire remote repository to your computer.
46. FORK

GitHub → Fork

OUTPUT:
A new repository is created under your GitHub account.
Meaning:
Creates your own GitHub copy of someone else's repository.
47. PULL REQUEST

GitHub → Pull Request → Create Pull Request

OUTPUT:
A Pull Request is created.
Meaning:
Requests that your changes be reviewed and merged into another branch/repository.
48. FINAL BASIC FLOW

LOCAL PROJECT
  ↓
git init
  ↓
git add .
  ↓
git commit
  ↓
git branch / git switch
  ↓
git merge
  ↓
git remote add origin
  ↓
git push
  ↓
GITHUB

GITHUB → LOCAL:

git fetch
  ↓
downloads changes

OR
git pull
↓
downloads + merges changes

49. IMPORTANT TERMINOLOGY — ONE LINE EACH

Git = Tool for tracking changes in code.
GitHub = Online platform for hosting Git repositories.
Repository = Project tracked by Git.
.git = Hidden folder containing Git information.
Working tree = Your actual project files.
Staging = Area containing changes ready for commit.
Commit = Saved snapshot of changes.
Branch = Separate line of development.
main = Common modern default branch.
master = Older/common branch name.
HEAD = Points to your current branch/commit.
Remote = Repository stored somewhere else.
origin = Default name of the remote repository.
add = Moves changes to staging.
commit = Saves staged changes.
push = Uploads local commits to remote.
pull = Downloads and merges remote changes.
fetch = Downloads remote changes without merging.
clone = Copies remote repository locally.
merge = Combines one branch into another.
switch = Changes the current branch.
restore = Restores a file to a previous committed state.
diff = Shows file differences.
log = Shows commit history.
fork = Creates your own GitHub copy of another repository.
Pull Request = Request to merge changes into another branch/repository.
conflict = When Git cannot automatically combine changes.
.gitignore = Tells Git which files to ignore.
tracked = File already managed by Git.
untracked = File Git has not started tracking.
staged = Change selected for the next commit.
-u = Sets the upstream branch.
-f = Forces an operation, especially a push.
Fast-forward = Merge where the target branch simply moves forward.
This is now in the single format you were asking for: COMMAND → OUTPUT → MEANING, all the way from configuring Git to pushing to GitHub.

From <https://chatgpt.com/>